

Rapport OptiPick — Optimisation de Tournées d'Entrepôt avec Coopération Humain-Robot

Module Programmation Logique et par Contraintes — Bachelor DATA & IA

Sommaire

Page	Section
1	Résumé (Abstract)
2	Problématique : comment nous nous y sommes pris
3	Matériel et méthode
4	Résultats : limites et performance
5	Conclusion : enseignements et perspectives
—	Bibliographie

Page 1 — Résumé (Abstract)

OptiPick est un système d'optimisation conçu pour la gestion de la préparation de commandes dans un entrepôt de e-commerce où coexistent différents types d'agents : des préparateurs humains, des robots autonomes et des chariots semi-autonomes guidés par des humains. Le projet vise à résoudre plusieurs défis combinés : l'allocation optimale des commandes aux agents, le respect des contraintes de capacité et de compatibilité (incompatibilités de produits, restrictions des robots), l'optimisation des tournées de picking (problème du voyageur de commerce, TSP), et la minimisation du coût opérationnel global. Nous avons développé une approche hybride combinant la modélisation par contraintes (MiniZinc), les solveurs d'optimisation (OR-Tools CP-SAT et Routing), et un moteur Python pour le chargement des données, la validation des contraintes et la visualisation. Les résultats montrent que l'allocation optimisée (MiniZinc, CP-SAT) réduit significativement la distance parcourue et le coût par rapport à une heuristique gloutonne First-Fit, tout en garantissant le respect des contraintes dures. Le projet inclut une interface web (Streamlit/Flask) déployée en ligne pour la visualisation en temps réel et cinq extensions avancées : picking multi-niveaux, gestion dynamique des commandes express, gestion des pannes et aléas, zones congestionnées, et intégration d'apprentissage par renforcement.

Page 2 — Problématique : comment nous nous y sommes pris

Contexte et objectifs

La problématique centrale est de préparer un ensemble de commandes clients de manière optimale en assignant chaque commande à un agent capable de la traiter, tout en respectant de nombreuses contraintes et en minimisant un score composite (distance, temps, coût). Les agents ont des capacités et restrictions différentes : les robots sont rapides et peu coûteux mais ne peuvent pas accéder à la zone réfrigérée, transporter des objets fragiles ou des articles de plus de 10 kg ; les humains sont flexibles mais coûteux ; les chariots nécessitent un humain dédié. De plus, certains produits sont incompatibles entre eux (chimiques vs alimentaires, électroniques vs détergents, etc.).

Approche méthodologique

Nous avons suivi une progression en six journées :

1. **Modélisation et allocation simple** : définition des classes (Warehouse, Product, Agent, Order), chargement des données JSON, calcul de distance Manhattan, allocation naïve First-Fit.
2. **Respect des contraintes dures** : vérification de capacité, incompatibilités, restrictions des robots, gestion des chariots ; intégration dans un module `constraints.py`.
3. **Optimisation des tournées (TSP)** : modélisation du problème du voyageur de commerce pour chaque agent, résolution via OR-Tools Routing, calcul du temps de tournée et vérification des deadlines.
4. **Allocation optimale et regroupement** : modélisation CSP avec OR-Tools CP-SAT et MiniZinc ; stratégie de batching (regroupement de commandes compatibles) ; comparaison First-Fit, MiniZinc, CP-SAT, Batching+CP-SAT.
5. **Optimisation du stockage et analyse** : analyse des patterns de commandes, réorganisation de l'entrepôt (produits fréquents près de l'entrée), simulation avant/après, analyse de coopération humain-robot.
6. **Interface web interactive** : application Flask et Streamlit avec visualisation en temps réel, animation des agents, formulaire d'ajout de commandes, choix de la méthode d'allocation.

Cette approche incrémentale nous a permis de valider chaque couche avant de passer à la suivante et d'obtenir un système complet et opérationnel.

Page 3 — Matériel et méthode

Environnement technique

- **Langage** : Python 3.8+
- **Moteur principal** : `main.py` orchestre le flux : chargement JSON → enrichissement des commandes → allocation (glouton ou optimisé) → calcul des métriques → rapport ou interface web.
- **Modules clés** : `loader.py` (parsing JSON), `models.py` (dataclasses), `constraints.py` (vérification), `allocation.py` (First-Fit et algorithmes gloutons), `allocation_cpsat.py` (OR-Tools CP-SAT), `minizinc_solver.py` (interface MiniZinc), `routing.py` (TSP avec OR-Tools), `batching.py` (regroupement de commandes).

MiniZinc

Le modèle principal `models/allocation.mzn` formalise le problème d'allocation comme un CSP/COP. Les variables de décision `assignment[order] = agent` indiquent quelle commande est assignée à quel agent (0 = non assignée). Les contraintes incluent : capacité poids/volume, zones interdites par agent, objets fragiles, poids maximal par item, incompatibilités entre commandes, et les cinq extensions (multi-niveaux, express, pannes/aléas, congestion, RL). La fonction objectif pondère le nombre de commandes assignées, la priorité express, et les pénalités. Les données sont injectées dynamiquement depuis Python via l'API `minizinc` ; il n'y a pas de fichier `.dzn` fixe car les données proviennent des JSON et varient à chaque exécution. Solveurs supportés : CBC, Gecode, Chuffed, HiGHS.

OR-Tools

- **CP-SAT** (`allocation_cpsat.py`) : résolution du problème d'allocation par programmation par contraintes entières. Variables binaires `x[i][j]` indiquant si la commande `i` est assignée à l'agent `j`. Contraintes de capacité, incompatibilités et restrictions. Objectif : minimiser coût ou distance.
- **Routing** (`routing.py`) : résolution du TSP pour calculer les tournées optimales de chaque agent. Utilise `create_distance_matrix()` (distance Manhattan) et `solve_tsp_with_ortools()` pour produire des séquences de collecte optimisées et vérifier les deadlines.

Flux de données

Les fichiers `warehouse.json`, `products.json`, `agents.json`, `orders.json` sont chargés par `loader.py`, enrichis (poids, volume, zones, fragilité, incompatibilités), puis passés soit à l'algorithme First-Fit, soit à MiniZinc ou CP-SAT. Les résultats sont des dictionnaires `{order_id: agent_id}` utilisés pour le calcul des métriques (distance, temps, coût) et l'affichage dans l'interface web.

Page 4 — Résultats : limites et performance

Synthèse des performances

L'évaluation a été réalisée sur un jeu de 10 commandes (données de test) avec 7 agents (3 robots, 2 humains, 2 chariots). Les métriques clés sont présentées ci-dessous.

Comparaison des méthodes d'allocation

Méthode	Commandes assignées	Distance (proxy)	Δ distance	Temps (min)	Coût (€)	Δ coût
First-Fit	10/10	182	—	15,3	3,82	—
MiniZinc	9/10	163	−10,4 %	13,1	2,18	−43 %
CP-SAT	9/10	167	−8,2 %	14,4	1,68	−56 %
Batching + CP-SAT	9/10	167	−8,2 %	14,8	0,74	−81 %

Interprétation :

- **First-Fit** (heuristique gloutonne) : très rapide, assigne toutes les commandes dans nos tests, mais sans optimisation globale. Distance et coût les plus élevés.
- **MiniZinc et CP-SAT** (solveurs exacts) : réduisent la distance de 8–10 % et le coût de 43–56 %. Une commande peut rester non assignée lorsque les contraintes sont trop strictes.
- **Batching + CP-SAT** : en regroupant les commandes compatibles, le coût chute à 0,74 € (−81 % par rapport à First-Fit) grâce à une meilleure utilisation des chariots et des robots.

Compromis heuristique vs solveur exact

Critère	Heuristique (First-Fit)	Solveur exact (MiniZinc, CP-SAT)
Temps de calcul	Très rapide	Plus lent
Qualité solution	Sous-optimale	Optimale ou proche
Complétude	Souvent toutes assignées	Peut laisser des non-assignées
Usage recommandé	Réactivité, gros volumes	Qualité, instances moyennes

Optimisation TSP

L'optimisation des tournées avec OR-Tools Routing réduit typiquement la distance de **15–20 %** par rapport à une simple estimation (somme des distances entrée ↔ emplacement). La fonction

check_deadlines garantit que les tournées respectent les contraintes temporelles des commandes.

Limites identifiées

1. **Évolutivité** : Pour de grandes instances (nombreux agents et commandes), MiniZinc et CP-SAT peuvent être lents ; il faut envisager des timeouts et un fallback vers First-Fit.
2. **Données synthétiques** : Les données utilisées sont simulées ; une validation sur flux réels d'entrepôt reste à réaliser.
3. **Modélisation** : La distance est une proxy (Manhattan) ; les temps de picking réels (ramassage, scan, manipulation) ne sont pas modélisés finement.
4. **Extensions** : Les extensions (multi-niveaux, congestion, RL) sont intégrées au modèle MiniZinc mais n'ont pas été évaluées quantitativement en termes de gain.
5. **Visualisation** : L'animation des agents est indicative et ne reflète pas un système de localisation en temps réel.

Heuristiques

Une heuristique est une méthode de résolution rapide d'un problème qui donne une bonne solution, mais pas forcément la solution optimale. Autrement dit : on simplifie le problème, on utilise des règles pratiques ou de l'intuition, et on obtient une solution acceptable rapidement.

Compromis algorithme exact vs heuristique :

Type	Résultat	Temps
Algorithme exact	Solution optimale	Souvent très lent
Heuristique	Bonne solution	Rapide

Exemples d'applications : recherche de chemin (GPS, A*), logistique (Amazon), jeux (échecs, Go), trading algorithmique, robotique.

10 heuristiques très utilisées en algorithmique :

1. **Greedy (glouton)** : on choisit la meilleure option locale à chaque étape. Exemples : rendu de monnaie, tâche la plus courte, ratio valeur/poids (sac à dos). Utilisé en : optimisation, graphes, compression (Huffman).
2. **Nearest Neighbor (plus proche voisin)** : toujours aller vers l'élément le plus proche. Très utilisé pour le TSP, la robotique, le GPS. Rapide mais pas toujours optimal.
3. **A*** (A-star) : recherche de chemin optimale. Formule $f(n) = g(n) + h(n)$ avec $g(n)$ = coût parcouru, $h(n)$ = estimation du coût restant (distance euclidienne ou Manhattan). Utilisé

dans : Google Maps, jeux vidéo, robotique.

4. **Branch and Bound** : calculer des bornes pour éliminer les solutions impossibles. Si un chemin coûte déjà plus cher que le meilleur trouvé → on arrête d'explorer. Utilisé en : optimisation combinatoire, TSP, programmation entière.
5. **Hill Climbing** : améliorer progressivement une solution (petite modification → garder si meilleur). Risque : optimum local. Utilisé en IA et optimisation.
6. **Simulated Annealing (recuit simulé)** : accepter des mauvaises solutions au début pour explorer, puis devenir plus strict. Évite les optimaux locaux. Utilisé en : optimisation industrielle, VLSI, logistique.
7. **Genetic Algorithms** : population de solutions, sélection, croisement, mutation. Utilisé en : IA, optimisation, finance, design industriel.
8. **Tabu Search** : mémoire des solutions explorées, empêcher les cycles. Utilisé en : planning, scheduling, logistique.
9. **Beam Search** : garder seulement les k meilleurs états. Utilisé en : NLP, traduction, génération de texte.
10. **Monte Carlo** : explorer des solutions aléatoirement, nombreuses simulations. Exemples : Monte Carlo Tree Search, AlphaGo, poker AI, robotique, trading.

Problèmes NP-complets

Non-deterministic Polynomial time (temps polynomial non déterministe) Temps polynomial veut dire que le temps de calcul augmente comme une puissance de la taille des données (n^2 , n^3 , etc.), et non de manière explosive (2^n , $n!$, etc.).

Exemple :

Taille des données : n (nombre de villes, nombre d'objets, etc.) Algorithme en temps polynomial : temps $\approx n$, n^2 ou n^3 → reste raisonnable même pour de gros n Algorithme en temps exponentiel : temps $\approx 2^n$ ou $n!$ → explose très vite quand n grandit Concrètement :

$n = 100 \rightarrow n^2 \approx 10\,000$ étapes (très rapide) $n = 100 \rightarrow 2^{100}$ étapes (inaccessible)

Voici 7 problèmes classiques NP-complets où l'on utilise presque toujours des heuristiques ou méta-heuristiques, car trouver la solution exacte devient impossible à grande échelle :

1. **Travelling Salesman Problem (TSP)**
2. **Knapsack Problem (Sac à dos)**
3. **Graph Coloring**

- 4. **Set Cover Problem**
- 5. **Vertex Cover**
- 6. **Job Scheduling**
- 7. **Bin Packing**

Méthodes d'optimisation avancées

- 1. **Mixed Integer Linear Programming (MILP)** — optimisation industrielle
- 2. **Monte Carlo Tree Search (MCTS)** — décisions complexes (ex. AlphaGo)
- 3. **Reinforcement Learning (RL)** — apprentissage par récompense et punition, stratégie adaptative
- 4. **Evolutionary Algorithms (Algorithmes génétiques)** — inspirés de l'évolution biologique.
Principe : générer une population de solutions, sélectionner les meilleures, croiser les solutions, muter
- 5. **Gradient-Based Optimization (Deep Learning)** — on minimise une fonction de coût $Loss(\theta)$ avec descente de gradient. Variantes modernes : SGD(Stochastic Gradient Descent), Adam, RMSProp

Algorithme	Domaine
MILP	Optimisation industrielle
Monte Carlo Tree Search	Décisions complexes
Reinforcement Learning	Stratégie adaptative
Algorithmes génétiques	Optimisation globale
Gradient descent	Machine learning

Optimiseurs de descente de gradient

3. SGD (Stochastic Gradient Descent)

SGD signifie descente de gradient stochastique. Au lieu d'utiliser tout le dataset, on met à jour les paramètres avec un seul exemple ou un petit batch.

Formule : $\theta = \theta - \alpha \nabla Loss(x_i)$

- **Avantages** : beaucoup plus rapide, permet d'entraîner des modèles très grands
- **Inconvénients** : plus de bruit dans l'apprentissage, convergence moins stable
- **Utilisé pour** : deep learning, réseaux neuronaux

4. RMSProp

RMSProp adapte automatiquement le learning rate pour chaque paramètre. Idée : si un gradient est souvent grand, on réduit son pas.

Formule simplifiée : $v_t = \beta v_{t-1} + (1 - \beta) g_t^2$; mise à jour : $\theta = \theta - \alpha g_t / \sqrt{v_t + \epsilon}$

- **Avantages :** stabilise l'apprentissage, fonctionne bien pour les réseaux profonds
- **Utilisé dans :** RNN, deep learning

5. Adam (Adaptive Moment Estimation)

Adam est aujourd'hui l'optimiseur le plus utilisé en deep learning. Il combine momentum et RMSProp. Il calcule la moyenne du gradient et la moyenne du gradient au carré.

Formules simplifiées : - $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ - $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ - mise à jour : $\theta = \theta - \alpha m_t / (\sqrt{v_t} + \epsilon)$

- **Avantages :** très stable, converge rapidement, peu de réglages
- **Optimiseur par défaut dans :** PyTorch, TensorFlow, Keras

Comparaison :

Algorithme	Caractéristique
SGD	Simple, rapide
RMSProp	Adapte le learning rate
Adam	Combine momentum + RMSProp

Compression de Huffman

La compression de Huffman est une méthode de compression sans perte. Elle permet de réduire la taille d'un fichier en utilisant moins de bits pour les symboles fréquents et plus de bits pour les symboles rares.

Principe : les caractères les plus fréquents obtiennent les codes binaires les plus courts.

Page 5 — Conclusion : enseignements et perspectives

Enseignements

- **Programmation par contraintes :** La modélisation en CSP/COP (MiniZinc, CP-SAT) est adaptée à l'allocation avec contraintes complexes (capacité, incompatibilités, restrictions). La séparation entre modèle et données facilite l'évolution et la maintenance.

- **Hybridation** : Combiner une heuristique rapide (First-Fit) pour les cas simples et des solveurs exacts (MiniZinc, CP-SAT) pour l'optimisation offre un bon compromis performance/qualité.
- **TSP et routing** : L'optimisation des tournées apporte un gain mesurable sur la distance ; OR-Tools Routing est facilement intégrable dans un pipeline Python.
- **Architecture modulaire** : Le découpage loader → models → constraints → allocation → routing permet des tests unitaires et l'ajout de nouvelles méthodes sans réécriture majeure.

Si l'on continuait le projet

1. **Validation sur données réelles** : Partenariat avec un entrepôt pour tester sur des flux de commandes et des configurations réelles.
2. **Optimisation multi-objectif** : Ponderer explicitement distance, temps, coût et équilibrage de charge avec des méthodes Pareto ou des poids configurables par l'utilisateur.
3. **Temps de résolution** : Fixer des timeouts, des objectifs de gap, et des stratégies de dégradation (fallback vers First-Fit si le solveur dépasse le temps imparti).
4. **Extensions opérationnelles** : Intégrer les pannes, la congestion et la dynamique des commandes dans un environnement de simulation continu pour évaluer la robustesse.
5. **Apprentissage par renforcement** : Pousser l'extension RL pour apprendre des politiques d'allocation adaptatives à partir de données historiques.
6. **Interface et déploiement** : Améliorer l'ergonomie de l'interface Streamlit, ajouter des tableaux de bord de suivi et une API REST pour une intégration avec des systèmes de gestion d'entrepôt (WMS).

En résumé, OptiPick démontre la faisabilité d'un système d'optimisation de tournées d'entrepôt fondé sur la programmation par contraintes et l'optimisation combinatoire, avec une base technique solide et des pistes claires pour une industrialisation future.

Bibliographie

Références écrites

- **MiniZinc** : Rossi et al., *MiniZinc 2.0: A declarative modeling language for constraint programming*, 2012
- **OR-Tools** : [Documentation officielle](#)
- **TSP** : Gutin & Punnen (eds.), *The Traveling Salesman Problem and Its Variations*, Springer, 2006

Vidéos YouTube — liens cliquables

#	Thème	Contenu	Vidéo
1	TSP	Problème du voyageur, OR-Tools Routing	► Regarder
2	Recuit simulé	Simulated annealing, TSP, Python	► Regarder
3	OR-Tools Routing	Routing solver	► Regarder
4	CP-SAT	Constraint Programming, Union Find	► Regarder
5	Recuit simulé (IA)	Recherche locale, simulated annealing	► Regarder
6	Algorithmes génétiques	Présentation des algorithmes génétiques	► Regarder
7	Reinforcement Learning	Introduction (Thibault Neveu)	► Regarder
8	Optimiseur Adam	Concept de momentum (Kevin Degila)	► Regarder
9	Codage de Huffman	Le codage de Huffman (Olivier Levêque)	► Regarder
10	MILP	Introduction à l'ILP, méthode graphique	► Regarder
11	RMSProp	Optimiseur RMSProp (DeepLearningAI)	► Regarder
12	Adam	Adam Optimization Algorithm (DeepLearningAI)	► Regarder

Projet OptiPick — Contributeurs : Nermine, Imen, Hamid, Romain